1

Fundamental Web API Patterns

Fundamental patterns and recommendations

Fundamental Patterns

1. **Resource Identification**
2. Standard Methods
3. Field Masks
4. Custom Methods
5. Long-running Operations
6. Re-runnable Jobs



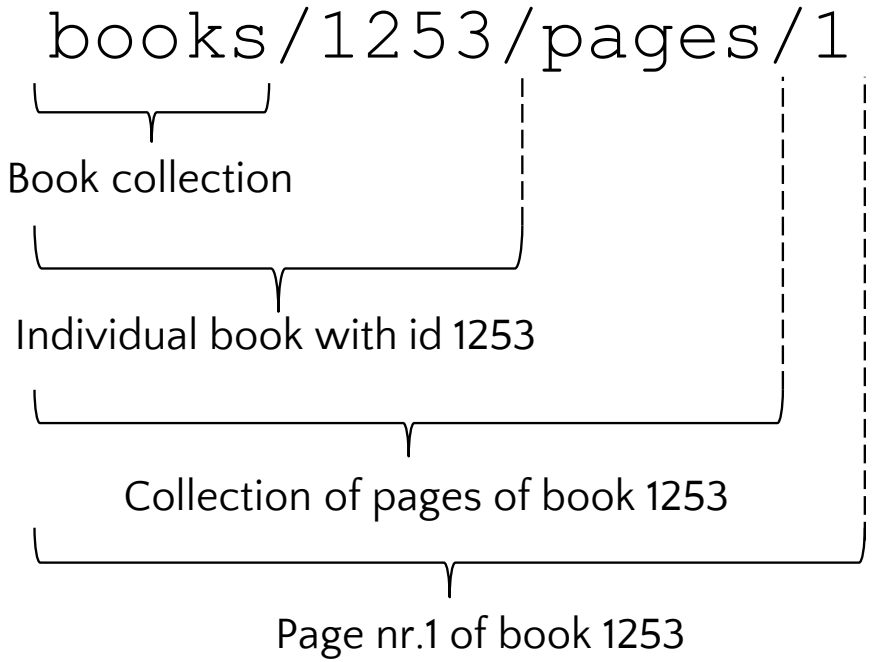
Resource Identification

- Resources are used to describe important concepts users may want to interact with.
  - Resources are grouped into collections. The collection name is the plural of a singular resource.
  - Individual resources are identified using appropriate identifiers, using the following format.
    - `<Collection name>/<resource id>`



Resource Identification

- Related resources can be hierarchically related.





Resource Identification

Identifiers should be:

- **Easy to use**
  - short, no special characters or punctuation (e.g. “/”, “á”))
- **Unique**
  - Unique on a certain relevant scope, e.g:
    - Across resources of same type
    - Across the entire application
    - Across the entire world…
- **Permanent** – Identifiers cannot change once assigned – even when the associated resource is deleted. Single-use, forever.
- **Fast and easily generated**



Resource Identification

Identifiers should be:

- **Unpredictable –** For security purposes, we may wish resource identifiers to be hard to identify.
- **Readable, shareable, verifiable** – Humans should be able to read, type and dictate identifiers reliably.
  - Avoid easily confused characters (eg: IlO0l|lIOI0Il)
  - Consider using checksums or similar approaches
- **Informationally dense** –
  - Decimal digits encode 3.3 bits of information.
  - Hexadecimal digits encode 4 bits of information per digit.
  - Base-32 encodes 5 bits of information per digit (e.g see Crockford's Base32 coding)



Resource Identification

Identifier Generation:

- Server-side generation
  - *Only way to ensure the desired properties*
- Random identifiers
- Tomb-Stoning – Preventing past identifiers being reused
  - Soft-deleting the entire resource
  - Soft-deleting the identifiers only
  - Using probabilistic data structures (Bloom Filter) to store past identifiers.
- Database:
  - Do not store checksums (compute and check on-the-fly)

Fundamental Patterns

1. Resource Identification
2. **Standard Methods**
3. Field Masks
4. Custom Methods
5. Long-running Operations
6. Re-runnable Jobs



Standard Methods

- One way to build a predictable API is to vary only the *resources* available while keeping a consistent set of specific actions (often called methods) that everyone can perform on those resources.
- *This pattern describes the expected methods and rules to be applied to each one.*



Standard Methods

- There are **six** standard methods, as presented here.

Name	Behavior
Get	Retrieves an existing resource
List	Lists a collection of resources
Create	Creates a new resource
Update	Updates an existing resource
Delete	Removes an existing resource
Replace	Replaces an entire resource



Standard Methods-General Considerations

- **When to implement? Do all resources require all standard methods?**
  - For consistency sake, if possible, then **yes**.
  - If not possible, then return an **`405 Method Not Allowed`** result.
    - *This should only be used if it is never possible to call the method on that type of resource.*
    - *If the method cannot be called on a specific instance of a resource, a* `403 Forbidden` *error should be returned instead.*



Standard Methods-General Considerations

- Standard methods should not have side effects.
- Some methods also require the property of **Idempotence:**

Idempotence

- Repeated calls of the same method with same parameters should have the same effect of a single call.



Standard Methods

GET

- The goal of the standard get method is very straightforward:
  - *The service has a resource record stored somewhere and this method returns the data stored in that record.*
- The method accepts only a single input: the identifier of the resource in question.
  - E.g. HTTP GET /books/123
- This method is expected to be idempotent.
  - *If no changes are made to the underlying resource, it always returns the same result.*



Standard Methods

LIST

- The goal of the standard list method is to list the resources of a target collection.
  - E.g HTTP GET /books
- Because more than one resource is involved, it may present issues that are not relevant to the single-resource GET method.
- This method is expected to be idempotent.



Standard Methods

LIST

- **Access Control:** What should the API do when the user does not have access to ALL the resources he is trying to list?
  - *The standard and expected approach is to return ALL the resources the user has access to, rather than forbidding the access.*



Standard Methods

LIST

- **Result Counts and Sorting:** Do **not** provide these unless explicitly asked for or necessary.
  - *Result counts do not scale well and can be hard to obtain if data is distributed.*
  - *If necessary, consider providing an* estimation *of the total size instead (and clearly identify it as such). This is usually more efficient to compute than actual count results.*
  - *Similar considerations apply to sorting (its hard and will overload the server).*



Standard Methods

LIST

- **Filtering :** Unlike counting and sorting, this is a common and useful feature, allowing wasteful processing to be avoided and bandwidth to be saved.
  - *Typical approach is passing the filter as a string to be processed by the API or the storage system itself.*



Standard Methods

CREATE

- Given some information about a resource, create that new resource in the API such that it can be retrieved by its identifier or discovered by listing resources.
  - `HTTP POST /books/ { title: Lusiadas, ... }`
- Always returns the newly created resource.



Standard Methods

CREATE

- The new resource id should almost always be generated by the server.
  - *In the rare occasion where it might make sense for the client to provide the key (e.g. synchronizing offline work), then the key will be embedded in the request itself*



Standard Methods

CREATE

- There is an expectation of *strong consistency* in API operation.
  - *Once a resource is created, it will be available for all other standard methods (GET, LIST, etc)*
- Scalable distributed databases operate on an *eventual consistency* model were information is propagated across multiple node.
  - *The newly created resource might not immediately appear in resource listings.*
  - *In this case, it is probably better to use a CUSTOM (non-standard) method for reading the data.*
  - *Alternatively, the Long Running Operation pattern might be used if the time until the resource is made available can be determined.*



Standard Methods

UPDATE

- The goal of this method is to change some existing information about a single resource.
  - Uses the HTTP PATCH method
  - Returns the modified resource.
- The method is expected to make partial changes rather than completely replacing the object:
  - Only the fields with specified changes should be modified.
  - This has some tricky issues (e.g. semantics of “don’t update” vs “blank” fields) will be examined at a later stage.



Standard Methods

UPDATE

- The standard UPDATE method is the preferred way of making changes to existing resources.
- However, in some cases, it might make more sense to update information as a collateral effect of some action
  - E.g. Setting the status of an email to “Sent” is better accomplished as a side-effect of sending the email in a custom function.



Standard Methods

DELETE

- The standard DELETE method is used to remove a previously created resource once it has outlived its usefulness.
  - Based on the HTTP DELETE method.
  - Unlike other methods, the return is void (No Content).



Standard Methods

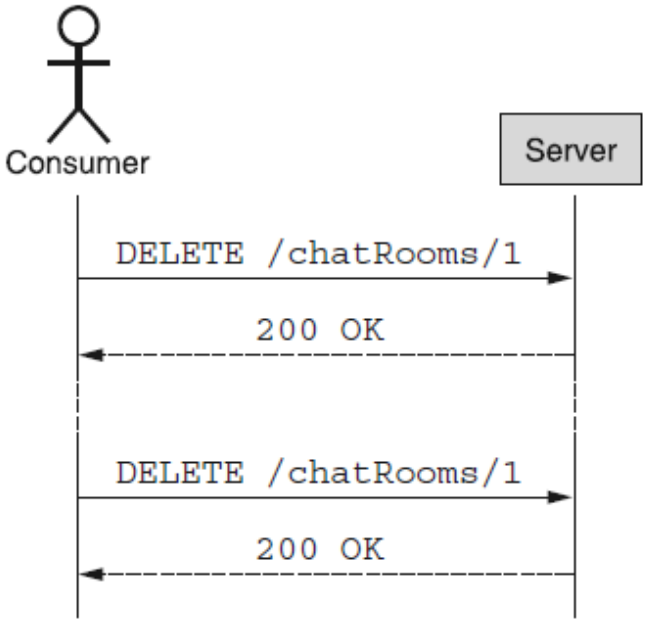
DELETE

- Idempotence and the DELETE method:
  - What is the expected behavior when there are multiple requests to remove a resource?
  - Declarative vs. imperative semantics!
    - Do you mean: “This resource should not exist”
    - Or: “Remove this resource”

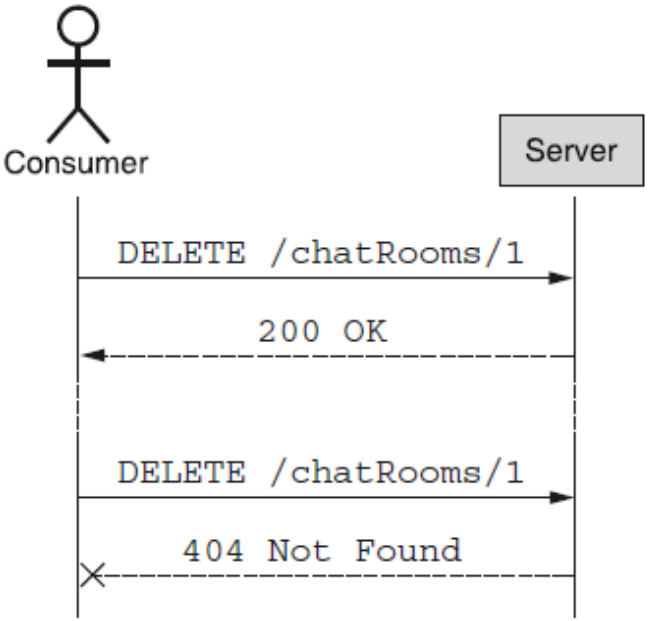
Standard Methods

Behavior should be aligned with the semantics of the API.
Imperative semantics are the most common by far.
In that case, idempotence is not achieved.

DELETE



Declarative Semantics
(Idempotence)



Imperative Semantics
(Not Idempotent)



Standard Methods

REPLACE

- A “semi-standard” method, it is used to update the entire contents of a resource, and not only conducting a partial update.
  - Based on the HTTP POST method.
- This may be helpful in some scenarios involving partial deletion of content.

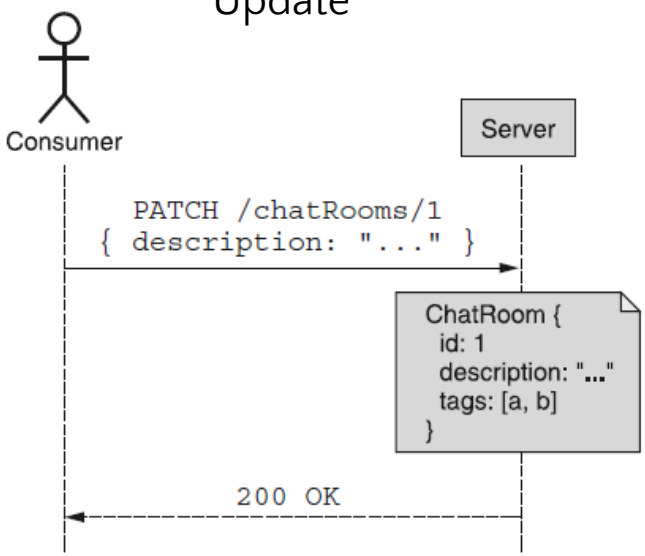


Standard Methods

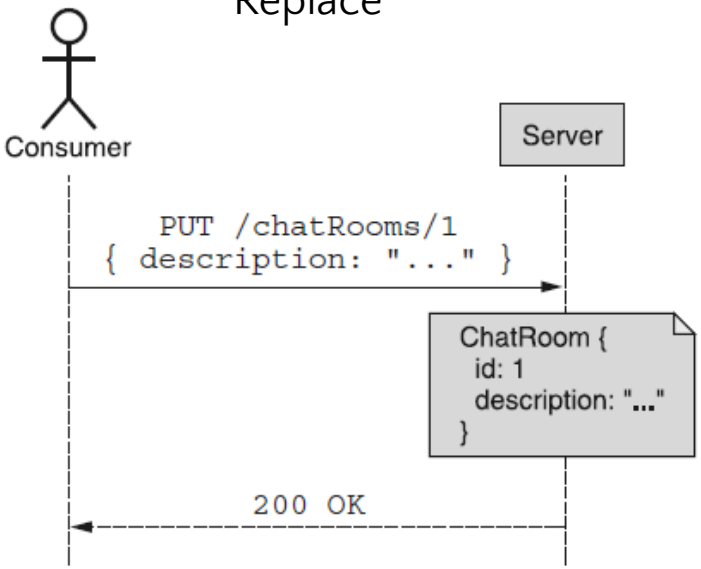
REPLACE

API consumer wants to erase tags and change description.
Update does not achieve this, but replace does.

Update



Replace





Standard Methods

REPLACE

- There is no way to know if the resource being REPLACEd existed previously.
- While REPLACE *may* be used to create a new resource, it does not replace CREATE:
  - CREATE will not create or modify an already existing resource.
  - REPLACE will always requires user-provided identifiers to be used.

Fundamental Patterns

1. Resource Identification
2. Standard Methods
3. **Field Masks**
4. Custom Methods
5. Long-running Operations
6. Re-runnable Jobs



Field Masks

- Sometimes, the API consumer wishes to retrieve or update only part of a resource.
  - *Partial retrievals are beneficial for performance reasons*
  - *Partial updated allow fine-grained replacement of objects.*

- Field masks allow such behavior to be possible.



Field Masks

- Total replacements may lead to unnecessary inconsistencies.

User 1 API call	User 2 API call

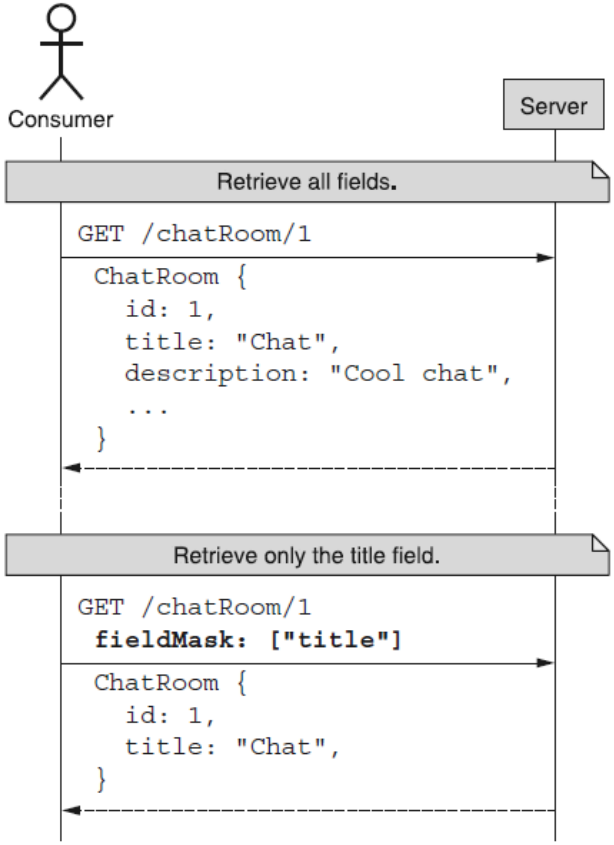
`let chatRoom = GetChatRoom({`
`  id: '1'`
`});`
 	
`chatRoom.title = "My Chat!";`
 	
`chatRoom = ReplaceChatRoom({`
`  chatRoom: chatRoom,`
`});`
	`let chatRoom = GetChatRoom({`
	`  id: '1'`
	`});`
	
	`let desc = fetchRandomQuote();`
	
	`chatRoom.description = desc;`
	
	`chatRoom = ReplaceChatRoom({`
	`  chatRoom: chatRoom,`
	`});`

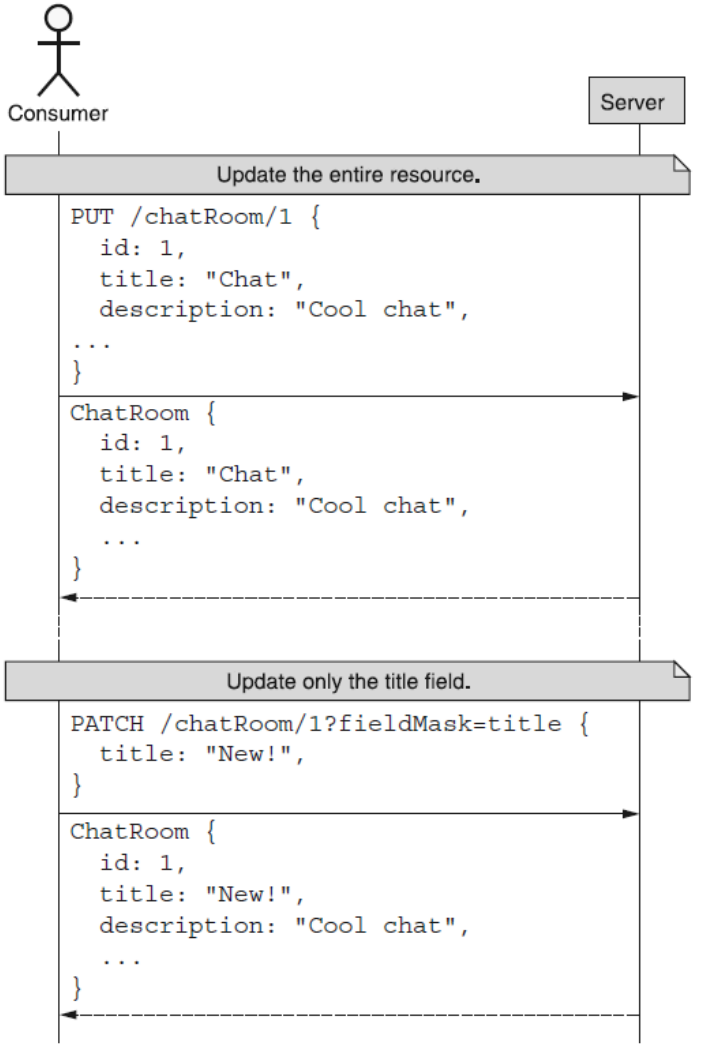
- Also, when API evolve, new fields may be added… Replacements for earlier API clients will simply erase the new fields.



Field Masks

- Field masks explicitly identify the relevant resources ins a request.

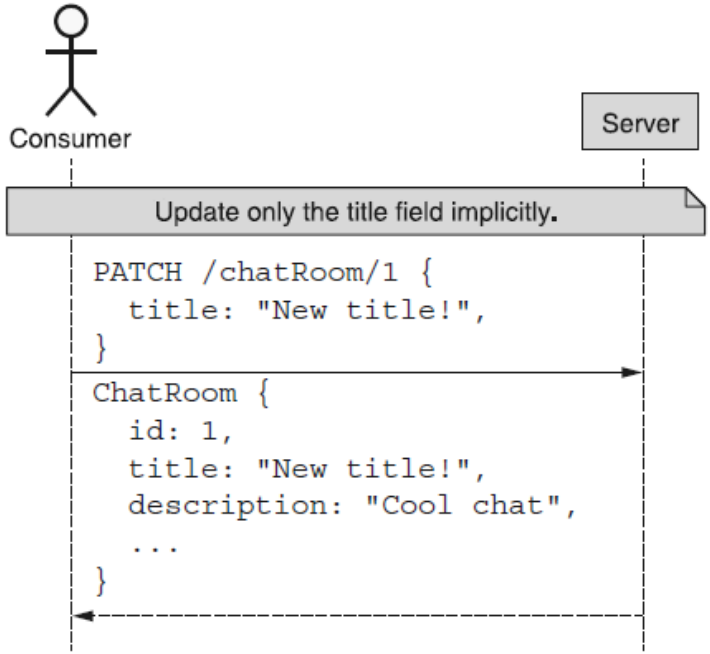






Field Masks

- Field masks can be *implicitly* inferred from specified parameters when not specified.





Field Masks

- Field masks cannot be passed in the HTTP request body:
  - GET does not have a body
  - PATCH has a body, but it should match the updated resource representation (consistency…)
- Two potential places:
  - HTTP Headers
  - Query Strings
- Query Strings are the most accessible and common option because they are visible in the request, unlike HTTP headers.
  - `/ChatRoom/1?fieldMask=title&fieldMask=description`



Field Masks

- Field masks cannot be passed in the HTTP request body:
  - GET does not have a body
  - PATH has a body, but it should match the updated resource representation (consistency…)
- Two potential places:
  - HTTP Headers
  - Query Strings
- Query Strings are the most accessible and common option because they are visible in the request, unlike HTTP headers.
  - `/ChatRoom/1?fieldMask=title&fieldMask=description`



Field Masks

- Field masks with nested data

ChatRoom resource	Field mask entry
{	"*"
"id": 1,	"id"
"title": "Chat Room",	"title"
"description": "Cool chat!"	"description"
"loggingConfig": {	"loggingConfig.*"
"maxSizeMb": 10,	"loggingConfig.maxSizeMb"
"maxMessageCount": 1000	"loggingConfig.maxMessageCount"
},	



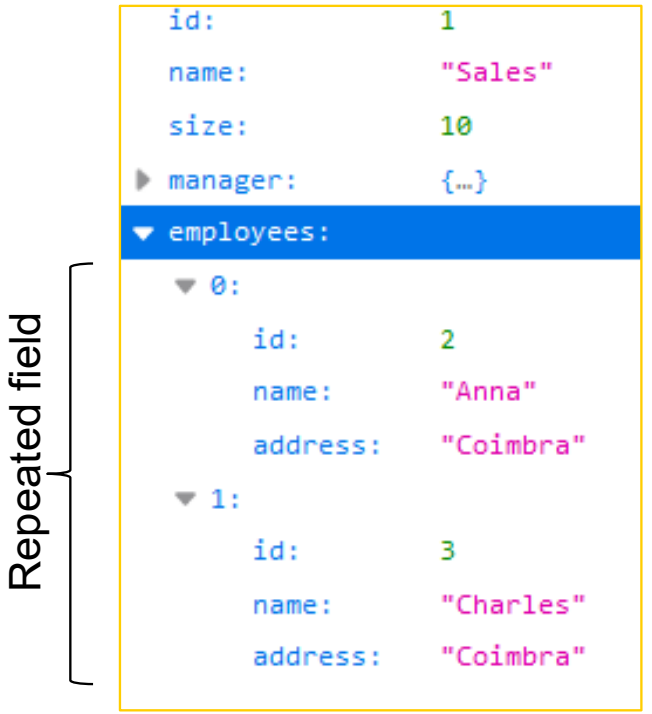
Field Masks

- Field masks with nested data

ChatRoom resource	Field mask entry
{	"*"
"id": 1,	"id"
"title": "Chat Room",	"title"
"description": "Cool chat!"	"description"
"loggingConfig": {	"loggingConfig.*"
"maxSizeMb": 10,	"loggingConfig.maxSizeMb"
"maxMessageCount": 1000	"loggingConfig.maxMessageCount"
},	

Field Masks

- Repeated fields and arrays
  - Do NOT assume that the order of data is (or will remain) always stable.
  - The API should NOT provide a means for accessing the “item at index 0”.
  - This significantly simplifies the evolution and maintenance of the API
- Data in repeated fields can be represented in field masks;
  - employees.* -> All employees
  - employees.*.name -> names of all employees





Field Masks

<host>/departments/1/


```
id:          1
name:        "Sales"
size:        10
manager:
  id:        1
  name:      "John"
  address:   "Lisbon"
employees:
  0:
    id:      2
    name:    "Anna"
    address: "Coimbra"
  1:
    id:      3
    name:    "Charles"
    address: "Coimbra"
```


<host>/departments/1/
?fieldMask=name&fieldMask=employees.*.name


```
id:          1
name:        "Sales"
employees:
  0:
    id:      2
    name:    "Anna"
  1:
    id:      3
    name:    "Charles"
```



Field Masks

- Default Values
  - The most sensible default value for the field mask is to retrieve all the fields.
  - In some rare occasions, the default might leave out some extensive/hard to compute fields.
    - This may lead to frustrating behavior, so it must be made very clear on the documentation
    - The special “*” symbol can be used to indicate that the user wants to retrieve ALL the fields (not only the default).

Return default fields (usually all)

```
<host>/departments/1/
```

Return ALL fields

```
<host>/departments/1/fieldMask=*
```



Field Masks

- Implicit mask values
  - Field masks for updating data cannot use the same defaults as for getting data
    - i.e. if default is all fields, then *update* becomes *replace*.
  - The natural default for updates is to change only the fields that are explicitly indicated in the request.
  - This provides an *implicit initialization* of field masks.
  - This is a very simple, but very powerful idea, that actually works well and allows other tricky issues to be solved…

```
PATCH /chatRooms/1 { "description": "New description" })

Field Mask is ["description"]
```



Field Masks

- Updating Dynamic Structures
  - In static structures, the fields remain constant.
    - A field may have an actual value or a null value.
    - Updating these is easy

```
PATCH /chatRooms/1 { "settings": { "test": "new value" } }
PATCH /chatRooms/1 { "settings": { "test": null } }
```

  - In dynamic structures, fields can change over time.
    - A field may have an actual value, a null value, or be undefined.

How should we update a dynamic data structure to make a field undefined?



Field Masks

- Options for undefining a field
  - Using “special” symbols for undefining a field is not standard and error prone.
  - Total replacement of the resource is possible, but this effectively becomes a REPLACE operation instead…
  - **The best approach: rely on an explicit field mask with missing value on the request body**

```
PATCH /chatRooms/1?fieldMask=settings.test HTTP/1.1
Content-Type: application/json
{}
```



Field Masks

- **Invalid fields**
  - Because APIs evolution frequently results in the adding or removing fields, it will not be uncommon to receive requests for missing fields, even for static resources.
  - The recommended approach is to ignore this request:
    - In fact, this is the same as assuming the API consumer is treating the resource as dynamic and requesting the undefinition of a field that is not defined).
    - *Providing an error instead would result in differentiated behavior for missing fields in dynamic and static resources. This would make the API unnecessarily complex and inconsistent.*